

GeoSPARQL: Spatial Data in RDF

Simon Frost

Overview

GeoSPARQL is an OGC standard for representing and querying spatial data in RDF. RDFLib.jl includes native support for:

- **WKT (Well-Known Text)** geometry parsing
- **Spatial predicates** (contains, within, intersects, disjoint, etc.)
- **Metric functions** (area, distance, centroid, convex hull)
- **SPARQL integration** via GeoSPARQL function URIs

```
using RDFLib
```

Creating Spatial Data

Geometries are stored as WKT literals with the `geo:wktLiteral` datatype:

```
geo = Namespace("http://www.opengis.net/ont/geosparql#")
ex = Namespace("http://example.org/")

g = RDFGraph()

# Add some spatial features
add!(g, Triple(ex.london, geo.hasGeometry, ex.london_geom))
add!(g, Triple(ex.london_geom, geo.asWKT,
    Literal("POINT(-0.1276 51.5074)",
        datatype=URIRef("http://www.opengis.net/ont/geosparql#wktLiteral"))))

add!(g, Triple(ex.paris, geo.hasGeometry, ex.paris_geom))
add!(g, Triple(ex.paris_geom, geo.asWKT,
    Literal("POINT(2.3522 48.8566)",
        datatype=URIRef("http://www.opengis.net/ont/geosparql#wktLiteral"))))
```

```

add!(g, Triple(ex.uk, geo.hasGeometry, ex.uk_geom))
add!(g, Triple(ex.uk_geom, geo.asWKT,
    Literal("POLYGON((-6 50, 2 50, 2 56, -6 56, -6 50))",
        datatype=URIRef("http://www.opengis.net/ont/geosparql#wktLiteral"))))

println("Graph has $(length(g)) triples")

```

Graph has 6 triples

Parsing WKT

The `parse_wkt` function converts WKT strings into geometry structs:

```

point = parse_wkt("POINT(-0.1276 51.5074)")
println("Point: ", point)

line = parse_wkt("LINESTRING(0 0, 1 1, 2 0)")
println("Line: $(length(line.points)) points")

polygon = parse_wkt("POLYGON((0 0, 10 0, 10 10, 0 10, 0 0))")
println("Polygon exterior: $(length(polygon.exterior)) points")

# SRID prefix is handled automatically
srid_point = parse_wkt("SRID=4326;POINT(2.3522 48.8566)")
println("SRID point: ", srid_point)

```

```

Point: GeoPoint(-0.1276, 51.5074, NaN)
Line: 3 points
Polygon exterior: 5 points
SRID point: GeoPoint(2.3522, 48.8566, NaN)

```

Spatial Predicates

Test topological relationships between geometries:

```

# A large bounding box and a point inside it
box = parse_wkt("POLYGON((-6 50, 2 50, 2 56, -6 56, -6 50))")
london = parse_wkt("POINT(-0.1276 51.5074)")
paris = parse_wkt("POINT(2.3522 48.8566)")

```

```
# Contains / Within
println("Box contains London: ", geo_contains(box, london))
println("Box contains Paris: ", geo_contains(box, paris))
println("London within box: ", geo_within(london, box))

# Intersects / Disjoint
println("Box intersects London: ", geo_intersects(box, london))
println("London disjoint Paris: ", geo_disjoint(london, paris))
```

```
Box contains London: true
Box contains Paris: false
London within box: true
Box intersects London: true
London disjoint Paris: true
```

Metric Functions

Compute geometric measurements:

```
# Area of a polygon
square = parse_wkt("POLYGON((0 0, 10 0, 10 10, 0 10, 0 0))")
println("Square area: ", geo_area(square))

# Distance between points
p1 = parse_wkt("POINT(0 0)")
p2 = parse_wkt("POINT(3 4)")
println("Distance: ", geo_distance(p1, p2))

# Area of a triangle
triangle = parse_wkt("POLYGON((0 0, 6 0, 3 6, 0 0))")
println("Triangle area: ", geo_area(triangle))
```

```
Square area: 100.0
Distance: 5.0
Triangle area: 18.0
```

Polygon with Holes

GeoSPARQL supports polygons with interior holes:

```

# Polygon with a hole
holed = parse_wkt("POLYGON((0 0, 20 0, 20 20, 0 20, 0 0), (5 5, 15 5, 15 15, 5 15, 5 5)))")
println("Outer ring: $(length(holed.exterior)) points")
println("Holes: $(length(holed.holes))")
println("Area: ", geo_area(holed)) # 400 - 100 = 300

# Point in the hole
center = parse_wkt("POINT(10 10)")
corner = parse_wkt("POINT(2 2)")
println("Center in holed polygon: ", geo_contains(holed, center)) # false (in hole)
println("Corner in holed polygon: ", geo_contains(holed, corner)) # true

```

```

Outer ring: 5 points
Holes: 1
Area: 300.0
Center in holed polygon: false
Corner in holed polygon: true

```

Multi-Geometries

```

# MultiPoint
mp = parse_wkt("MULTIPOINT((0 0), (1 1), (2 2))")
println("MultiPoint: $(length(mp.points)) points")

# MultiPolygon
mpoly = parse_wkt("MULTIPOLYGON(((0 0, 1 0, 1 1, 0 1, 0 0)), ((2 2, 3 2, 3 3, 2 3, 2 2)))")
println("MultiPolygon: $(length(mpoly.polygons)) polygons")
println("Total area: ", geo_area(mpoly))

```

```

MultiPoint: 3 points
MultiPolygon: 2 polygons
Total area: 2.0

```

SPARQL Integration

GeoSPARQL functions can be used directly in SPARQL FILTER expressions via the `geof:` namespace:

```

# Build a graph with spatial data
g2 = RDFGraph()
places = [
    (ex.london, "POINT(-0.1276 51.5074)"),
    (ex.paris, "POINT(2.3522 48.8566)"),
    (ex.berlin, "POINT(13.4050 52.5200)"),
    (ex.madrid, "POINT(-3.7038 40.4168)"),
]

for (place, wkt) in places
    geom_node = URIRef(string(place) * "_geom")
    add!(g2, Triple(place, geo.hasGeometry, geom_node))
    add!(g2, Triple(geom_node, geo.asWKT,
        Literal(wkt, datatype=URIRef("http://www.opengis.net/ont/geosparql#wktLiteral"))))
end

# Query: find all places with their WKT
results = sparql_query(g2, """
    PREFIX geo: <http://www.opengis.net/ont/geosparql#>
    SELECT ?place ?wkt
    WHERE {
        ?place geo:hasGeometry ?geom .
        ?geom geo:asWKT ?wkt .
    }
    """)
for row in results
    println(" $(split(string(row["place"]), '/') [end]): $(row["wkt"])")
end

```

```

london: Literal("POINT(-0.1276 51.5074)", datatype=URIRef("http://www.opengis.net/ont/geosparql#wktLiteral"))
paris: Literal("POINT(2.3522 48.8566)", datatype=URIRef("http://www.opengis.net/ont/geosparql#wktLiteral"))
berlin: Literal("POINT(13.4050 52.5200)", datatype=URIRef("http://www.opengis.net/ont/geosparql#wktLiteral"))
madrid: Literal("POINT(-3.7038 40.4168)", datatype=URIRef("http://www.opengis.net/ont/geosparql#wktLiteral"))

```

Boundary Computation

Compute the boundary of a geometry:

```

poly = parse_wkt("POLYGON((0 0, 10 0, 10 10, 0 10, 0 0))")
boundary = geo_boundary(poly)
println("Polygon boundary: ", typeof(boundary), " with $(length(boundary.points)) points")

```

```

line = parse_wkt("LINESTRING(0 0, 5 5, 10 0)")
lb = geo_boundary(line)
println("Line boundary: MultiPoint with $(length(lb.points)) endpoints")

```

Polygon boundary: GeoLineString with 5 points

Line boundary: MultiPoint with 2 endpoints

Summary

RDFLib.jl's GeoSPARQL support enables spatial reasoning over RDF data:

Feature	Function
Parse WKT	<code>parse_wkt(s)</code>
Contains	<code>geo_contains(a, b)</code>
Within	<code>geo_within(a, b)</code>
Intersects	<code>geo_intersects(a, b)</code>
Disjoint	<code>geo_disjoint(a, b)</code>
Distance	<code>geo_distance(a, b)</code>
Area	<code>geo_area(g)</code>
Boundary	<code>geo_boundary(g)</code>
Buffer	<code>geo_buffer(g, dist)</code>

All spatial functions work with Points, LineStrings, Polygons, and their Multi- variants, following the OGC Simple Features specification.